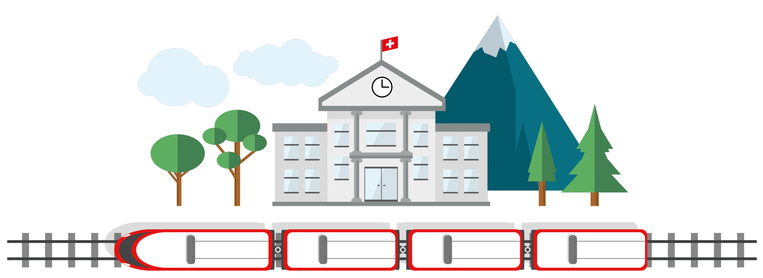
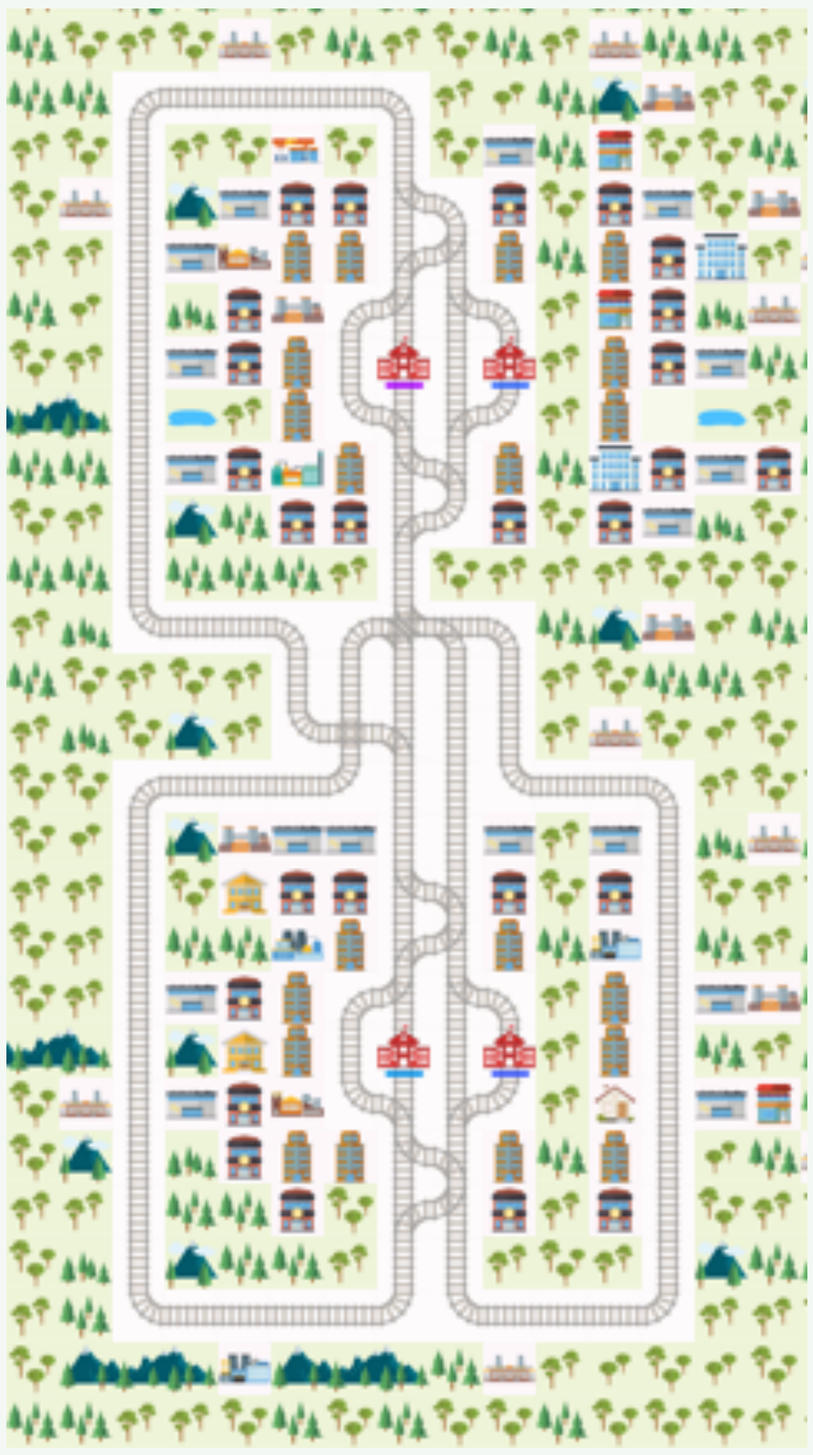
# Flaspland

## Railway routing and scheduling using Answer Set Programming

Ryan Kepler J. Murphy
University of Potsdam



## Flatland Environment



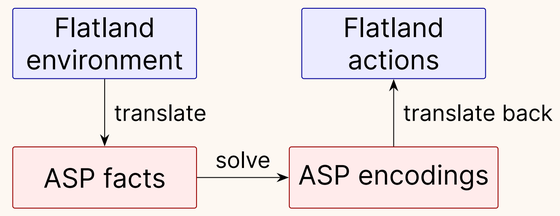
Flatland is a virtual railway scheduling environment designed for a global railway scheduling competition. It provides a grid-based rail network on which multiple trains must be routed and scheduled while avoiding conflicts, subject to track topology, speed profiles, and random malfunctions.

**Why Flatland?** It offers lightweight, reproducible benchmark instances that isolate the core combinatorial challenges of real-world railway operations — *routing*, *scheduling*, and *conflict resolution* — making it an ideal target for declarative, ASP-based approaches. Additionally, Flatland provides a mechanism for visualizing solutions, which is invaluable both for debugging and for comparing different solution strategies.

## Translation Pipeline

The logical flow is as follows:

1. Flatland generates an environment instance
2. This instance is translated into ASP facts
3. A set of *clingo* encodings computes a solution
4. The result is translated back into Flatland actions

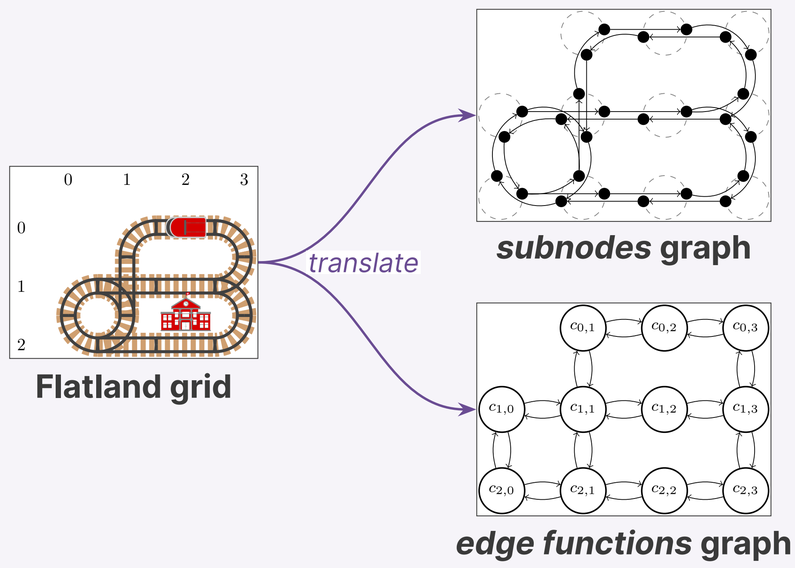



**Example environment instance (excerpt):**

```
train(3).
start(3,(8,8),1,n). end(3,(21,9),156). speed(3,3).
...
cell((19,9), 49186).   cell((19,10), 34864).
cell((20,9), 34864).   cell((20,10), 32872).
cell((21,9), 32800).   cell((21,10), 32800).
cell((22,9), 37408).   cell((22,10), 49186).
cell((23,9), 32872).   cell((23,10), 37408).
cell((24,9), 49186).   cell((24,10), 34864).
```

## From Flatland Grid to Flaspland Graph

We convert the Flatland grid into directed graphs and consider two graph encodings — *subnodes* and *edge functions* — that differ in how cells and directions are represented.




Each graph has follows a different approach for preserving train direction and ensuring proper behavior of the trains. The *subnodes* graph leaves direction implicit, while the *edge functions* graph explicitly defines directional conditions.

## Future Work

### Features

- Malfunctions and rescheduling
- Intermediate stations
- Energy consumption
- Environment representation

### Techniques

- Temporal logic
- Large neighbourhood search
- Timestep-free partial ordering
- Prioritized planning

### Find Out More

University of Potsdam